

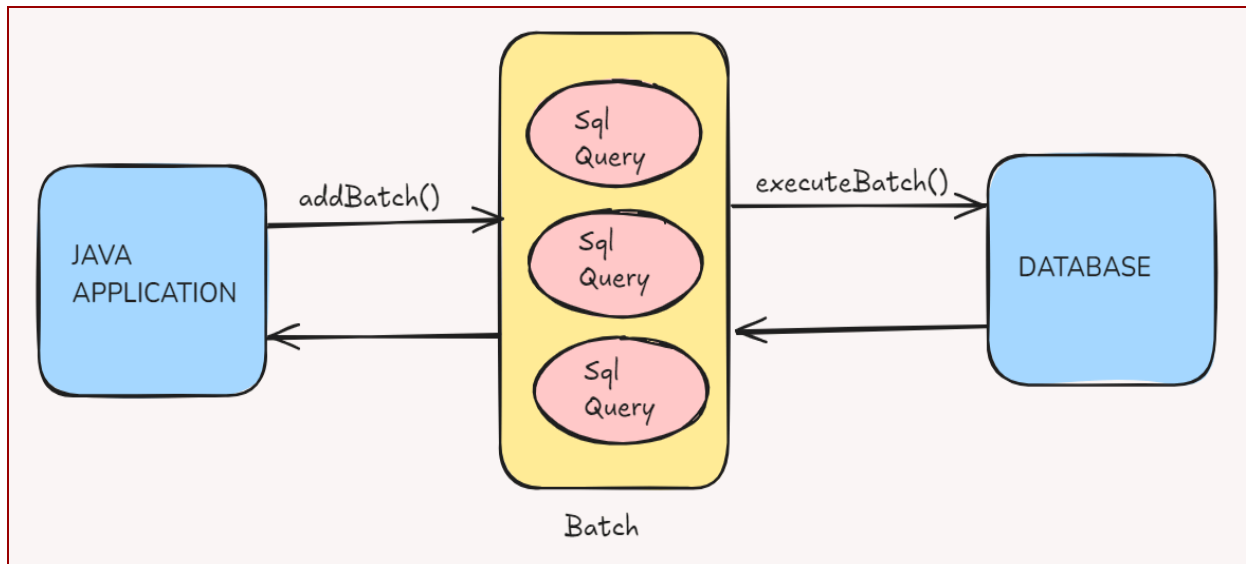
20 - Batch Update

Batch Update in JDBC allows to grouping of multiple SQL queries together and executing them as a single batch.

- It optimize database interactions by reducing the number of trips between the application and the database server.
- Instead of executing each query separately, we can add several similar or identical queries to a batch and execute them at once.
- Batch updates can be used with both Statement and PreparedStatement objects.
- It can be **applied to non-select operations** like Insertion, Updation, and Deletion.

Why Use Batch Update?

- **Efficiency:**
 - In typical JDBC operations, each SQL query is executed independently, which causes multiple database round-trips.
 - With batch updates, you reduce these round-trips, improving overall application performance, especially for large-scale operations like inserting multiple records.
- **Bulk Operations:**
 - When performing bulk operations such as mass data insertion or updating/deleting many records, batch updates streamline the process by allowing multiple similar operations to be executed simultaneously.
- **Reduced Network Traffic:**
 - Sending multiple queries in one go decreases the load on both the application and database by minimizing the number of interactions between them in the database.



👉 Methods Used in Batch Update:

- **addBatch():**

This method adds SQL commands to a batch, allowing multiple queries to be added sequentially.

- **executeBatch():**

This method executes all the queries added in the batch at once. It returns an array of integers representing the number of rows affected by each query in the batch.

👉 Steps to Perform Batch Update:

1. Batching SQL Queries:

- You add multiple SQL queries (typically of the same type) to a batch using the addBatch() method.
- This method adds the SQL query to the batch but does not execute it immediately.

```
String sql = "UPDATE studentinfo SET sage=? WHERE id=?";
```

```
pstmt = connect.prepareStatement(sql);
```

```
pstmt.setInt(1, 20);
```

```
pstmt.setInt(2, 1);
```

```
pstmt.addBatch();
```

```
pstmt.setInt(1, 24);
```

```
pstmt.setInt(2, 5);
```

```
pstmt.addBatch();
```

```
pstmt.setInt(1, 16);
```

```
pstmt.setInt(2, 44);
```

```
pstmt.addBatch();
```

2. Executing the Batch:

- Once all SQL queries are added to the batch, the `executeBatch()` method is invoked.
- This sends all queries in the batch to the database for execution at once.

```
int[] rowsAffected = pstmt.executeBatch();
```

3. Handling Results:

- After executing the batch, `executeBatch()` returns an array of integers.

- Each element in the array corresponds to the number of rows affected by each SQL query in the batch.

```
for (int i : rowsAffected) {  
    System.out.println("Rows affected: " + i);  
}
```

Code:

```
public class BatchUpdateExample {  
  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/studentdb";  
        String username = "root";  
        String password = "password";  
  
        String sql = "UPDATE studentinfo SET sage=? WHERE id=?";  
  
        try (  
            Connection connect = DriverManager.getConnection(url, username,  
password);  
            PreparedStatement pstmt = connect.prepareStatement(sql);  
        ) {  
            pstmt.setInt(1, 20);  
            pstmt.setInt(2, 1);  
            pstmt.addBatch();  
  
            pstmt.setInt(1, 24);  
            pstmt.setInt(2, 5);  
            pstmt.addBatch();  
  
            pstmt.setInt(1, 16);
```

```
pstmt.setInt(2, 44);  
pstmt.addBatch();  
  
int[] rowsAffected = pstmt.executeBatch();  
  
for (int i : rowsAffected) {  
    System.out.println("Rows affected: " + i);  
}  
  
} catch (SQLException e) {  
    e.printStackTrace();  
}  
}  
}
```